Data Structures

MIDIGIWORLD

Introduction to Data Structures

- A data structure is a way of storing data in a computer so that it can be used efficiently and it will allow the most efficient algorithm to be used.

- A data structure should be seen as a logical concept that must address two fundamental concerns.

  I. How the data will be stored

  II. what operations will be performed on it.

Classification of Data Structures

Simple and Compound Data Structures

- **Simple Data Structure:** Simple data structure can be constructed with the help of primitive data types. A primitive data structure used to represent the standard data types of any one of the computer languages. Variables, pointers, structures, unions, etc. are examples of simple data structures.

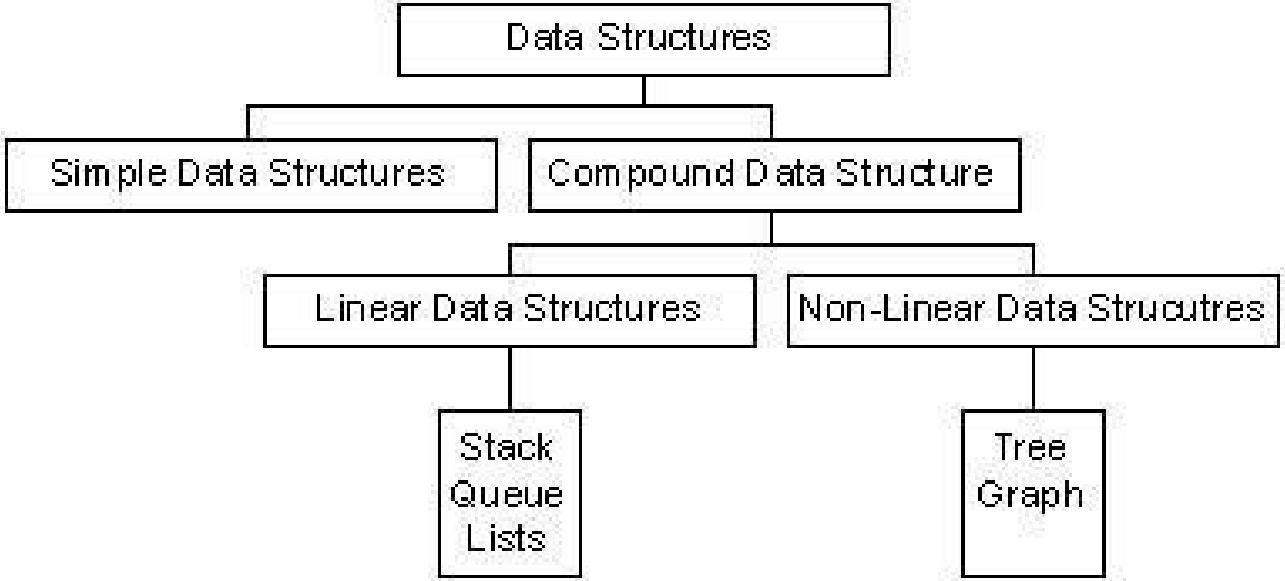
- **Compound Data structure:** Compound data structure can be constructed with the help of any one of the primitive data types and it is having a specific functionality. It can be designed by user. It can be classified as

  - Linear data structure
  - Non-linear data structure

Classification of Data Structures

Data structures can be classified as

- Simple data structure
- Compound data structure



MIDIGIWORLD

Classification of Data Structures

Linear and Non-linear Data Structures

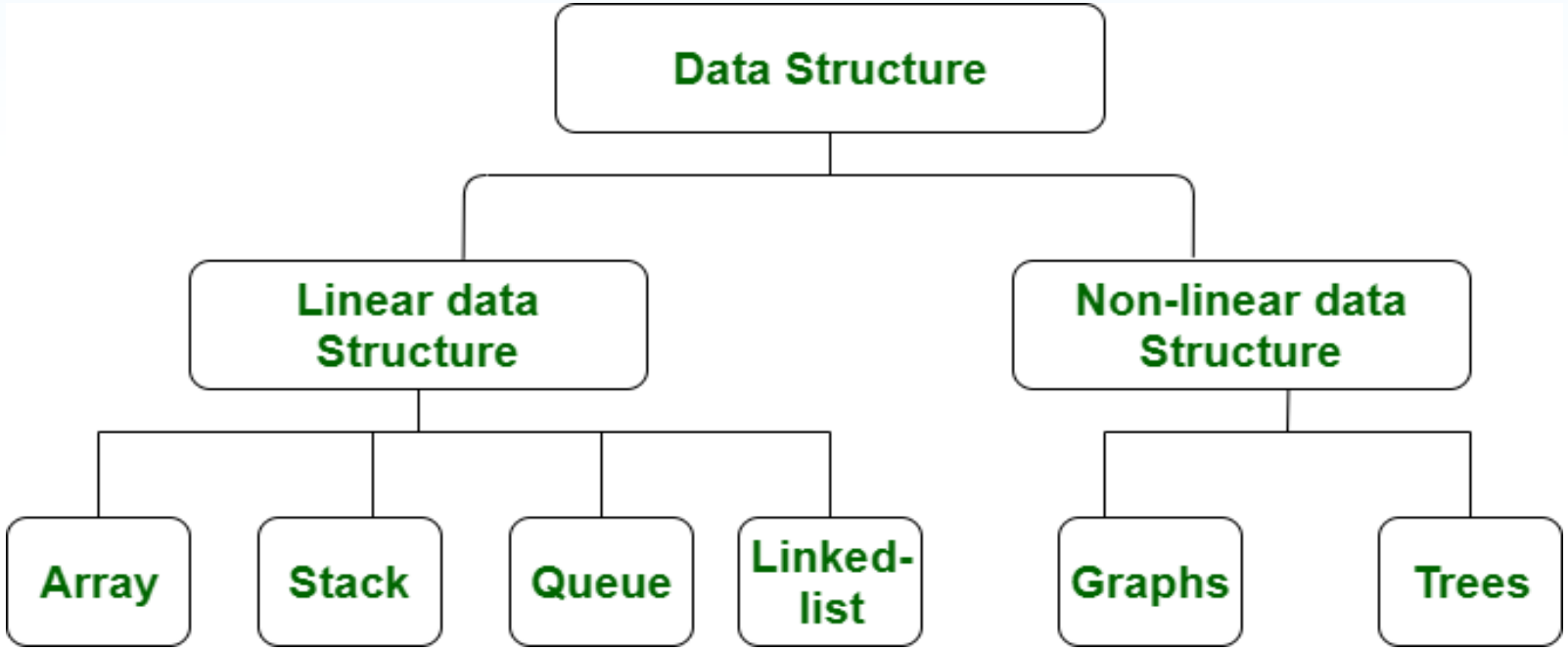
- **Linear Data Structure:** Linear data structures can be constructed as a continuous arrangement of data elements in the memory.

  Ex : Arrays, Linked List

- **Non- Linear Data Structure:** Non-Linear data structures can be constructed as a collection of randomly distributed set of data item joined together by using a special pointer (tag).

  Ex :Tree, Graph

Classification of Data Structures



MIDIGIWORLD

Basic Operations on Data Structures

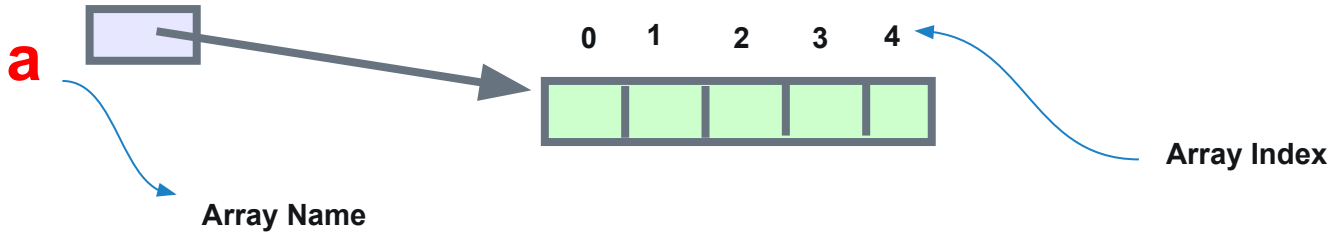
- Add an element
- Delete an element
- Traverse / Display
- Sort the list of elements
- Search for a data element

Linear Data Structure

I. Arrays

- An array is a fixed-size sequential collection of elements with same data types that share a common name.
- It is simply a group of data types.
- An array is a derived data type.
- An array is used to represent a list of numbers or a list of names.

Ex: int a[5];



Linear Data Structure-Arrays

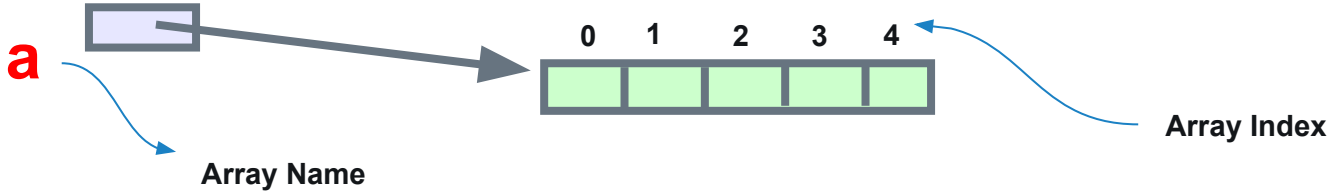
Types of Arrays

❖ There are three types of arrays.

1. One Dimensional Array
2. Two Dimensional Array
3. Multi Dimensional Array

1. One Dimensional Arrays-1D

A variable which represent the list of items using only one index (subscript) is called one-dimensional array.



MIDIGIWORLD

Linear Data Structure-Arrays

One Dimensional Arrays-1D

The general form of array declaration is :

type array-name[size];

- Here the type specifies the data type of elements contained in the array, such as int, float, or char.
- And the size indicates the maximum numbers of elements that can be stored inside the array.
- The size can be either a numeric constant or a symbolic constant.

Example:

Array Size

Array Data Type

int number[5];

Array Name

MIDIGIWORLD

Linear Data Structure-Arrays

One Dimensional Arrays-1D

For Example ,

if we want to represent a set of five numbers say (35,40,20,57,19) by an array variable number, then number is declared as follows

```
int number [5]={35,40,20,57,19} ;
```

The values can be assigned to the array as follows :

```
number [0] = 35;
number [1] = 40;
number [2] = 20;
number [3] = 57;
number [4] = 19;
```

Linear Data Structure-Arrays

Initialization Of One-dimensional Arrays :

An array can be stored in following stages :

1. At compile time
2. At run time

Compile time initialization :

I. In compile time initialization, the array is initialized when they are declared.
II. The general form of initialization of array is :
    type array-name[size] = { list of values };
III. The list of values are separated by commas.

Ex:

```
int number[]= {10,20,30,40,50};
int number[5]= {10,20,30,40,50};
```

MIDIGIWORLD

Linear Data Structure-Arrays

Compile time initialization:

- If the number of values in the list is less than the number of elements, then only that many elements will be initialized.
- The remaining elements will be set to zero automatically

Ex:

```
int number[5]= {10,20,30};
```

in this case,

number[0] will get 10
number[1] will get 20
number[2] will get 30
but,
number[3] will get 0
number[4] will get 0

Linear Data Structure-Arrays

Run time initialization :

- The array is explicitly initialize at run time.
- This concept generally used for initializing large arrays.

Ex:

```
int number[50];

for(i=0;i<50;i++)
 {
     printf("Enter value for a[%d]:",i);
     scanf("%d",&number[i]);
 }
```

- This segment helps to get data for 50 time from the user at run time.
- During Compile time all 50 blocks will be initiated with garbage values.

Linear Data Structure-Arrays

Two-dimensional Arrays-2D

- A variable which represent the list of items using two index (subscript) is called two-dimensional array.
- In Two dimensional arrays, the data is stored in **rows and columns** format.
- The general form of two dimensional array declaration is :

  Datatype *array-name[row_size][column_size];*

- Here the type specifies the data type of elements contained in the array, such as int, float, or char.
- The size should be either a numeric constant or a symbolic constant.

Example:

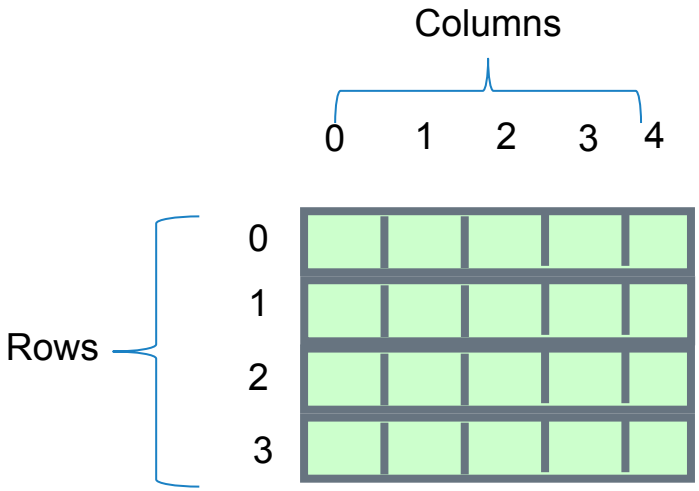
Int table[2][3];

MIDIGIWORLD

Linear Data Structure-Arrays

Two-dimensional Arrays-2D:

- In Two dimensional array memory representation, the data is stored in **rows and columns** format.



Linear Data Structure-Arrays

Two-dimensional Arrays-2D

An array can be stored in following stages :

1. At compile time
2. At run time

Compile time initialization :

I. In compile time initialization, the array is initialized when they are declared.

II. The general form of initialization of array is :

type array-name[size][size] = { list of values };

III. The list of values are separated by commas.

IV. Using the above format data can be stored in TWO ways.

Linear Data Structure-Arrays

Two-dimensional Arrays-2D

Format -1:

```
int table[2][3] = {1,2,3,4,5,6};
```

Format -2:

```
int table[2][3] = {{1,2,3},{4,5,6}};
```

- Here the elements of first row initializes to zero and the elements of second row initializes to one.

The memory layout of above example :

```
table[0][0] = 1;
table[0][1] = 2
table[0][2] = 3;
```

```
table[1][0] = 4;
table[1][1] = 5;
table[1][2] = 6;
```

MIDIGIWORLD

Linear Data Structure-Arrays

Two-dimensional Arrays-2D

- If the values are missing in size, they are automatically set to ONE.

  Example:

  int table[][3] = {{1,2,3},{4,5,6}};

  Here the size of first row initializes to one and then based on the number of elements size will be fixed during runtime.

- If the values are missing in an initializer, they are automatically set to zero.

  Example:

  int table[2][3] = {1,2,3};

  Here first row initialize to 1,2 and 3, and second row initialize to 0,0 and 0 automatically.

Linear Data Structure-Arrays

Basic Operations On Arrays

- On Arrays the following operations can be performed
  1. **Insertion**
  2. **Deletion**
  3. **Searching**
  4. **Sorting**
  5. **Merging**
  6. **Display**
- Using arrays we can able create and use STACK and QUEUE data structures
- STACK and QUEUE are called as derived data structures because implemented from arrays.

MIDIGIWORLD

Linear Data Structure-Stack

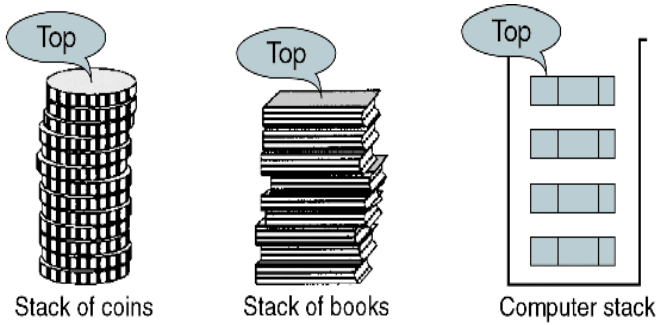
What is a Stack?

- A stack is a data structure of ordered items such that items can be inserted and removed only at one end.
- A stack is a LIFO (Last-In/First-Out) data structure
- A stack is sometimes also called a pushdown store.

Basic Operations On Stacks:

- On Arrays the following operations can be performed

1. **PUSH/ Insertion**
2. **POP/ Deletion**
3. **Display**



MIDIGIWORLD

Linear Data Structure-Stack

When we create or using STACK data structure, the following steps to be completed before performing any operations.

- STACK size to be fixed first.
- ‘top’ is a pointer will tells where to insert or delete data from the STACK
- So ‘top’ pointer should be initiated with -1
- Before we try to insert data into STACK, We need to check for STACK Overflow condition, means check for availability of free block in STACK
- Before try to remove data from STACK , we need to check for STACK Underflow condition, means check for availability of DATA in STACK
- At any point of time only one data can be inserted or removed from STACK
- When we insert data into STACK, then ‘top’ pointer should be incremented by 1.
- When we remove data from STACK, the ‘top’ pointer should be decremented by 1.

Linear Data Structure-Stack

What can we do with a stack?

- push - place an item on the stack
- peek - Look at the item on top of the stack, but do not remove it
- pop - Look at the item on top of the stack and remove it

What are some applications of stacks?

- Conversion & Evaluating Expressions
- Program Execution (OS)
- Recursion

Linear Data Structure-Stack

Applications of stacks- Conversion & Evaluating Expressions

- Why Conversion required in computer:

Infix expressions are **readable and solvable by humans**. We can easily distinguish the order of operators, and also can use the parenthesis to solve that part first during solving mathematical expressions. **The computer cannot differentiate the operators and parenthesis easily**, that's why conversion is needed.

- There are three type of expressions

1. Infix
2. Prefix
3. Postfix

- Expression conversion process can be applied only on Arithmetic Operators.

Ex. +, -, *, /, %

MIDIGIWORLD

Linear Data Structure- Stack

Applications of stacks- Conversion & Evaluating Expressions

- Why do we have two more notations when we have infix expression notation?

  infix notation requires the additional symbols. The order of operations within prefix and postfix expressions is completely determined by the position of the operator

- Infix- Operator comes between two operands – ex. A+B
- Prefix- Operator comes before two operands- ex. +AB
- Postfix- Operator comes after two operands- ex. AB+

- Why is stack used in expression conversion?

  In STACK **Operators will be stored based their precedence**. **High precedence operators will be kept on top and low precedence operators will be in bottom**. So that **high precedence operators will be executed first**.

MIDIGIWORLD

Linear Data Structure- Stack

Applications of stacks- Conversion & Evaluating Expressions

1. Converting infix to postfix expression

Example

Infix Expression	Postfix Expression
A + B	A B +
A + B * C	A B C * +
(A + B) * C	A B + C *

MIDIGIWORLD

Applications of stacks- Conversion & Evaluating Expressions

Steps to be followed in Converting infix to postfix expression

- Scan input string from left to right character by character.
- If the character is an operand, put it into output array.
- If the character is an operator and operator's stack is empty, push operator into operators' stack.
- If the operator's stack is not empty, there may be following possibilities.

1. If the precedence of scanned operator is greater than the top most operator of operator's stack, push this operator into operand's stack.
2. If the precedence of scanned operator is less than or equal to the top most operator of operator's stack, pop the operators from operand's stack until we find a low precedence operator than the scanned character. Never pop out (**'('**) or (**')'**) whatever may be the precedence level of scanned character.
3. If the character is opening round bracket (**'('**), push it into operator's stack.
4. If the character is closing round bracket (**')'**), pop out operators from operator's stack until we find an opening bracket (**'('**).
5. Now pop out all the remaining operators from the operator's stack and push into output array.

Applications of stacks- Evaluating Expressions

Steps in Postfix Expression Evaluation using Stack

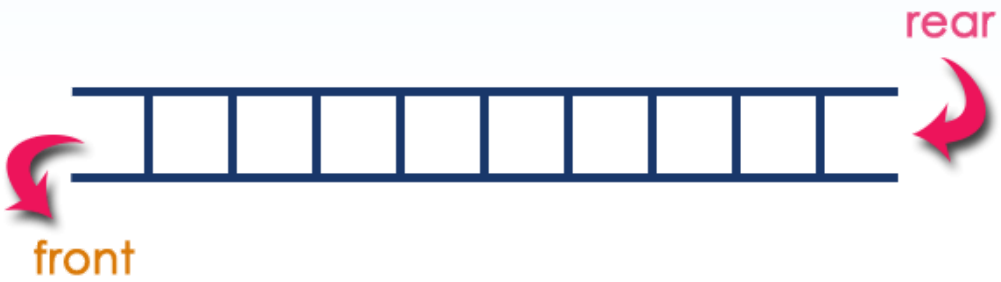
A postfix expression can be evaluated using the Stack data structure. To evaluate a postfix expression using Stack data structure we can use the following steps.

- **Read all the characters one by one from left to right in the given Postfix Expression**
- **If the reading symbol is operand, then push it on to the Stack.**
- **If the reading symbol is operator (+ , - , * , / etc.,), then perform TWO pop operations and store the two popped operands in two different variables (operand1 and operand2). Then perform reading symbol operation using operand1 and operand2 and push result back on to the Stack.**
- **Finally! perform a pop operation and display the popped value as final result.**

Linear Data Structure-Queue

What is Queue?

Queue is a linear data structure in which the operations are performed at two different ends.



The insertion operation is performed at a position which is known as '**rear**' and the deletion operation is performed at a position which is known as '**front**'.

Queue data structure is a linear data structure in which the operations are performed based on FIFO principle.

Linear Data Structure-Queue

Operations on a Queue

- enQueue(value) - (To insert an element into the queue)
- deQueue() - (To delete an element from the queue)
- display() - (To display the elements of the queue)
- Queue data structure can be implemented in two ways. They are as follows...

  1. **Using Array**
  2. **Using Linked List**

- When a queue is implemented using an array, that queue can organize an only limited number of elements.
- When a queue is implemented using a linked list, that queue can organize an unlimited number of elements.

Linear Data Structure-Queue

Types of Queues

- Based on Implementation and usage, the queues are classified as follwoes,
  1. Normal Queue
  2. Circular Queue
  3. Priority Queue
  4. Double ended Queue (Dqueue)

Applications of Queues

- Queue is used when things don’t have to be processed immediately.

Example

1. When a resource is shared among multiple consumers. Examples include CPU scheduling, Disk Scheduling.

Linear Data Structure-Queue

Applications of Queues

2) When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.

3) In Operating systems:

a) Semaphores
b) FCFS (first come first serve) scheduling, example: FIFO queue
c) Spooling in printers
d) Buffer for devices like keyboard

4) In Networks:

a) Queues in routers/ switches
b) Mail Queues

Linear Data Structure-Queue

Implementation of Queue

- The following steps to followed during the implementation of Normal Queue
  - The size of the queue and data type for the queue should be fixed.
  - To perform insertion **rear** pointer should be initiated to -1 and **front** pointer should be fixe with 0.
  - To insert data into queue, we need to check for condition rear==Size-1

    1. If rear==size-1 is true, We can say that “ Queue is Over flow”.
    2. If the above condition is false, then there is a free block to insert data into queue.
    3. To find free block increment by 1 with current rear value- (rear=rear+1)
    4. We can use new rear value to locate in queue to store data (que[rear]= data)
    5. From setp-1 to 4 we can repeat till we fill all blocks of memory in queue.

Linear Data Structure-Queue

Implementation of Queue

- To delete data from queue, we need to check for condition front>rear

  1. If front>rear is true, We can say that “ Queue is UnderFlow”.
  2. If the above condition is false, then there is a data to delete from queue.
  3. Using front pointer value we can find the location to perform delete operation (n= que[front])
  4. After remove data from queue, we need to incremented front value by 1
  5. From setp-1 to 4 we can repeat till we remove all data from queue.

- To display data from queue, we need to check for condition front>rear

  1. If front>rear is true, We can say that “ Queue is Empty”.
  2. if the above condition fails, we can display data from queue using current front pointer position to current rear pointer position.

Linear Data Structure-Queue

Circular Queue

- Circular queue is implemented to over come the disadvantages of Normal Queue.
- The disadvantage of Normal Queue is wasting of memory.
- For example

1. The queue after inserting all the elements into

Queue is Full

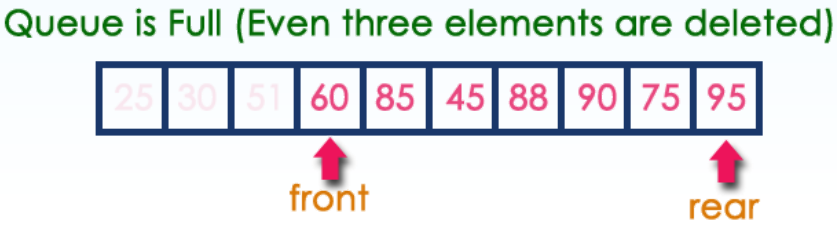
2. Now consider the following situation after deleting three elements from the queue.

Queue is Full (Even three elements are deleted)

MIDIGIWORLD

Linear Data Structure-Queue

Circular Queue



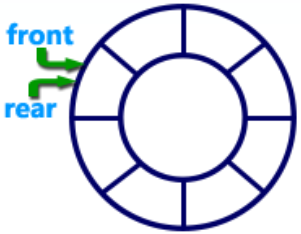
3. After removing three elements from Queue, the Queue is Full and we cannot insert the new element because '**rear**' is still at last position

- Even though we have empty positions in the queue we can not make use of them to insert the new element. This is the major problem in a normal queue.
- To overcome this problem we use a circular queue data structure.

Linear Data Structure-Queue

What is Circular Queue

- A circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle.



- In Circular queue operations are not depend on Front and Rear pointers.
- Front and Rear Pointers are used to find the block of memory to perform operation of insert and delete, Front and Rear pointers are initiated with-1.
- In Circular queue one more pointer called Count will be used to decide operations are possible on queue or not. Counter pointer will be initiated with 0.

Linear Data Structure-Queue

Operations on Circular Queue

- To insert data into circular queue, we need to check for Count== Size
  1. If the above condition is true, that means we cannot perform insert operation and we can say that “ Queue is Overflow”
  2. If the above condition is fails, that means their will be free block to insert data into circular queue.
  3. To find free block we need to find rear value using rear= (rear+1)%Size
  4. After finding rear, we can insert data into circular queue using current rear.
  5. After inserting data, Count pointer should incremented by 1
  6. We can repeat from step-1 to 5, when ever there is a block available for data to store.

Linear Data Structure-Queue

Operations on Circular Queue

- To remove data from circular queue, we need to check for Count== 0

1. If the above condition is true, that means we cannot perform delete operation and we can say that “ Queue is Underflow”

2. If the above condition is fails, that means there are some data in circular queue to delete.

3. To find location to delete data from queue we need to find front value using front=(front+1)%Size

4. After finding front, we can delete data from circular queue using current rear.

5. After inserting data, Count pointer should decremented by 1

6. We can repeat from step-1 to 5, when ever there is a block available for data to store.

MIDIGIWORLD

Dynamic Memory Allocation

MIDIGIWORLD

Dynamic Memory Allocation

Problem with Linear Data Structures

- Sometimes Amount of data cannot be predicted beforehand Number of data items keeps changing during program execution.
- Wasteful of memory space.

Better Solution is - **Dynamic memory allocation**

- Know how much memory is needed after the program is run
- Dynamically allocate only the amount of memory needed
- C provides functions to dynamically allocate memory
  malloc, calloc, realloc, free

Dynamic Memory Allocation

Memory Allocation Functions

malloc()

Allocates requested number of bytes and returns a pointer to the first byte of the allocated space

calloc()

Allocates space for an array of elements, initializes them to zero and then returns a pointer to the memory.

free()

Frees previously allocated space.

realloc()

Modifies the size of previously allocated space.

Most Frequently used functions are malloc() and free().

Dynamic Memory Allocation

malloc()

- A block of memory can be allocated using the function malloc
- Reserves a block of memory of specified size and returns a address (pointer)
- The return pointer can be type-casted to any pointer type
- The allocation can fail if sufficient memory space is not available
- If it fails, malloc returns NULL

General format or Syntax:

```
type *p;
p =  (type *) malloc (byte_size);
```

MIDIGIWORLD

Dynamic Memory Allocation

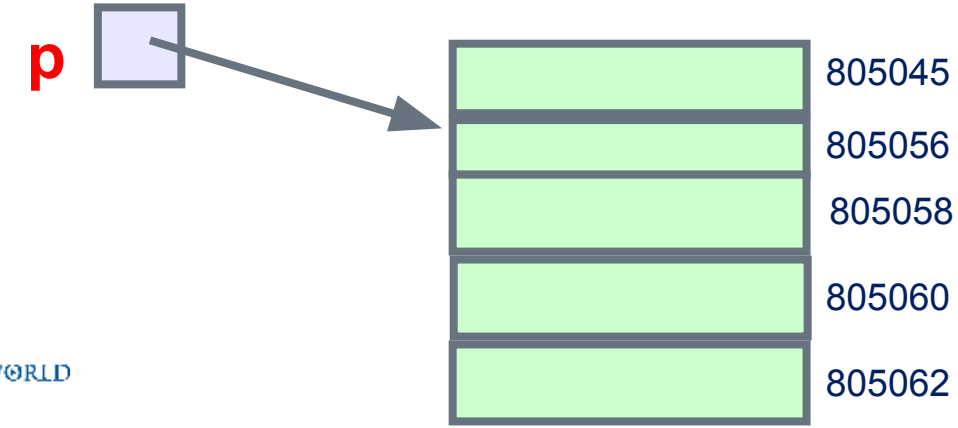
malloc()

Syntax- for single block of memory:

```
type *p;
p =  (type *) malloc (byte_size);
```

Ex: int *p;

```
p=(int*)malloc(size_of(int));
```



MIDIGIWORLD

Dynamic Memory Allocation

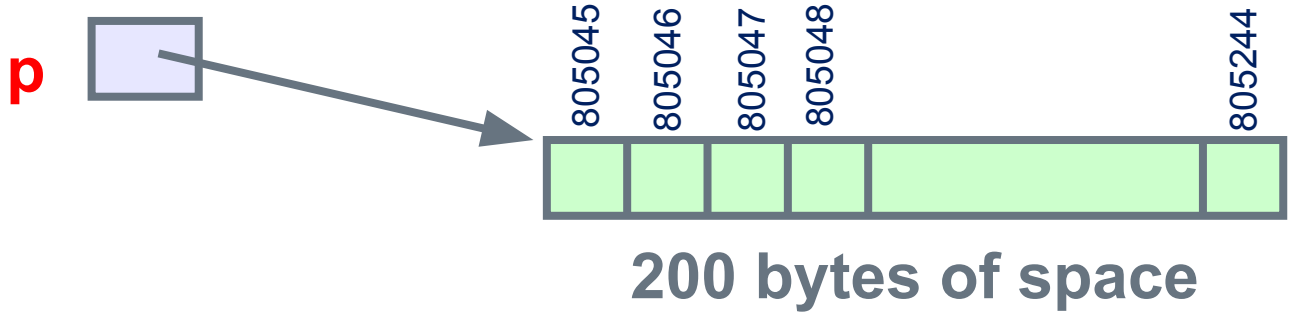
malloc()

Syntax- for multiple block of memory:

```
type *p;
p =  (type *) malloc (n*byte_size);
```

Ex: int *p, n=100;

```
p=(int*)malloc(n*size_of(int));
```



Dynamic Memory Allocation

free()

An allocated block can be returned to the system for future use by using the free function

General syntax:

```
free (p);
```

where p is a pointer to a memory block which has been previously created using malloc

Linear Data Structure- Linked List

What is Linked List?

- The linked list is a linear data structure that contains a sequence of elements such that each element links to its next element in the sequence.

- Each element in a linked list is called "Node".
- “Node” – Created with help of malloc function
- Linked list are created in dynamically so there is no wastage of memory.

Linear Data Structure-Linked List

Types of Linked List

- Based on structure of the Nodes, there are three type of linked list
  1. Single Linked List
  2. Circular Linked List
  3. Double Linked List

Operations on Linked List

- Following operations are possible on linked list
  1. Inserting a node in list
  2. Deleting a node from list
  3. Traversal
  4. Merging different list

Linear Data Structure-Linked List

Single Linked List

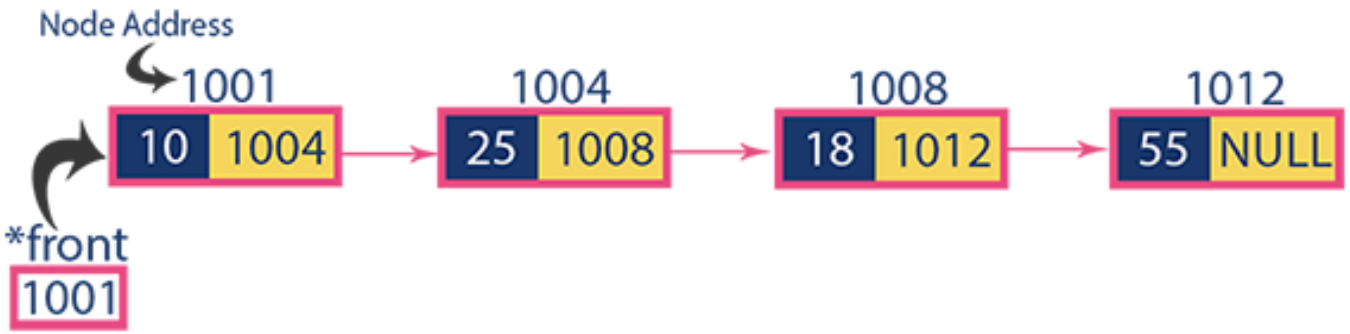
- Single linked list is a sequence of elements in which every element has link to next element in the sequence.
- In any single linked list, the individual element is called as "Node".
- Every "Node" contains two fields, data field, and the next field.
- The data field is used to store actual value of the node and next field is used to store t address of next node in the sequence.
- The graphical representation of a node in a single linked list is as follows..

Linear Data Structure-
Linked List

Single Linked List

- In a single linked list, the address of the first node is always stored in a reference node know as "front" (Some times it is also known as "head“ or “First”).
- Always the last node must be NULL.

Example



Linear Data Structure-Linked List

Operations on Single Linked List

The following operations are performed on a Single Linked List

- **Insertion**
- **Deletion**
- **Display**

1. Insertion

- In a single linked list, the insertion operation can be performed in three ways. They are as follows...
  1. Inserting At Beginning of the list
  2. Inserting At End of the list

Linear Data Structure- Linked List

Operations on Single Linked List- Insertion

1. Inserting At Beginning of the list

❖ We can use the following steps to insert a new node at beginning of the single linked list

Step 1 - Create a **newNode** with given value.

Step 2 - Check whether list is **Empty** (**head == NULL**)

Step 3 - If it is **Empty** then, set **newNode→next = NULL** and **head = newNode**.

Step 4 - If it is **Not Empty** then, set **newNode→next = head** and **head = newNode**.

2. Inserting At End of the list

❖ We can use the following steps to insert a new node at end of the single linked list

Step 1 - Create a newNode with given value and newNode → next as NULL.

Step 2 - Check whether list is Empty (head == NULL).

Step 3 - If it is Empty then, set head = newNode.

Linear Data Structure- Linked List

Operations on Single Linked List- Insertion

2. Inserting At End of the list

Step 4 - If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.

Step 5 - Keep moving the **temp** to its next node until it reaches to the last node in the list (until **temp** → **next** is equal to **NULL**).

Step 6 - Set **temp** → **next** = **newNode**.

Linear Data Structure-Linked List

Operations on Single Linked List- Insertion

Deletion

- In a single linked list, the deletion operation can be performed in two ways. They are as follows.

  1. Deleting from Beginning of the list
  2. Deleting from End of the list

Deleting from Beginning of the list

- We can use the following steps to delete a node from beginning of the single linked list...

Step 1 - Check whether list is **Empty** (**head** == **NULL**)
Step 2 - If it is **Empty** then, display **'List is Empty!!! Deletion is not possible'** and terminate the function.
Step 3- else move **head->link to head (head= head->link)**

Linear Data Structure- Linked List

Operations on Single Linked List- Insertion

Deleting from End of the list

❖ We can use the following steps to delete a node from end of the single linked list.

Step 1 - Check whether list is **Empty** (**head == NULL**)

Step 2 - If it is **Empty** then, display **'List is Empty!!! Deletion is not possible'** and terminate the function.

Step 3 - If it is **Not Empty** then, define two Node pointers **'temp1'** and **'temp2'** and initialize **'temp1'** with **head**.

Step 4 - Check whether list has only one Node (**temp1 → next == NULL**)

Step 5 - If it is **TRUE**. Then, set **head = NULL** and delete **temp1**. And terminate the function. (Setting **Empty** list condition)

Step 6 - If it is **FALSE**. Then, set **'temp2 = temp1** ' and move **temp1** to its next node. Repeat the same until it reaches to the last node in the list. (until **temp1 → next == NULL**)

Step 7 - Finally, Set **temp2 → next = NULL** and delete **temp1**.

Linear Data Structure- Linked List

Operations on Single Linked List- Insertion Display

❖ We can use the following steps to display the elements of a single linked list.

Step 1 - Check whether list is **Empty** (**head** == **NULL**)

Step 2 - If it is **Empty** then, display **'List is Empty!!!'** and terminate the function.

Step 3 - If it is **Not Empty** then, define a Node pointer **'temp'** and initialize with **head**.

Step 4 - Keep displaying **temp** → **data** with an arrow (**--->**) until **temp** reaches to the last node

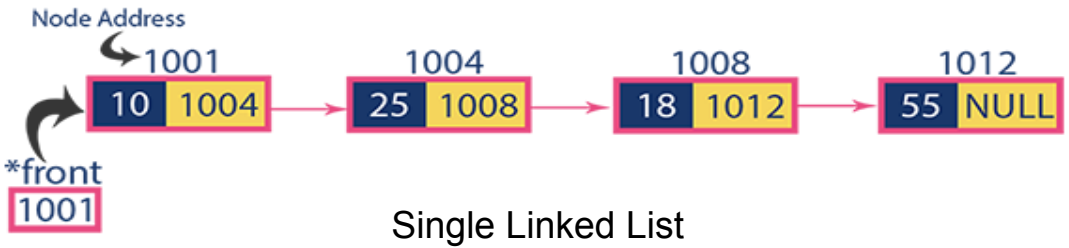
Step 5 - Finally display **temp** → **data** with arrow pointing to **NULL** (**temp** → **data ---> NULL**).

Linear Data Structure-Linked List

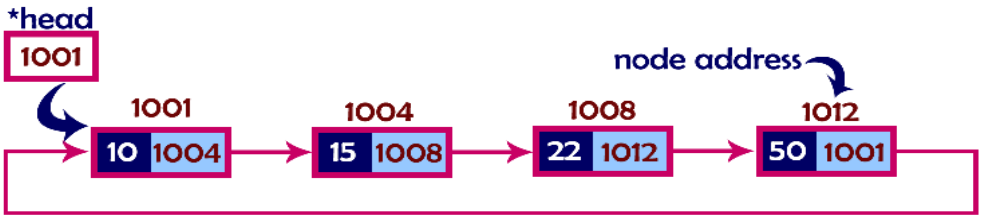
Circular Linked List

- A circular linked list is a sequence of elements in which every element has a link to its next element in the sequence and the last element has a link to the first element.

Example:



Single Linked List



Circular Linked List

Linear Data Structure-Linked List

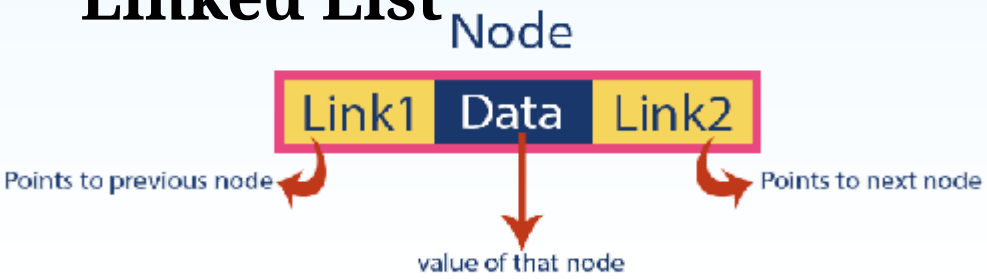
Double Linked List

- **Double linked list is a sequence of elements in which every element has links to its previous element and next element in the sequence.**

Need of Double Linked List:

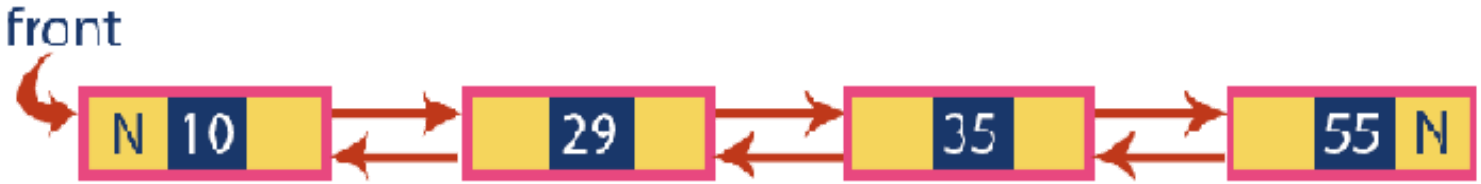
- In a single linked list, every node has a link to its next node in the sequence.
- we can traverse from one node to another node only in one direction and we can not traverse back.
- In a double linked list, every node has a link to its previous node and next node. So, we can traverse forward by using the next field and can traverse backward by using the previous field.
- Every node in a double linked list contains three fields

Linear Data Structure-Linked List



Here, **'link1'** field is used to store the address of the previous node in the sequence, **'link2'** field is used to store the address of the next node in the sequence and **'data'** field is used to store the actual value of that node.

Example



Linear Data Structure-Linked List

Double Linked List

❖ **Important Points to be Remembered**

1. In double linked list, the first node must be always pointed by head.
2. Always the previous field of the first node must be NULL.
3. Always the next field of the last node must be NULL.

❖ **Operations on Double Linked List**

In a double linked list, we perform the following operations...

1. Insertion
2. Deletion
3. Display

Linear Data Structure-Linked List

Double Linked List

❖ **Insertion**

In a double linked list, the insertion operation can be performed in two ways as follows...

1. Inserting At Beginning of the list
2. Inserting At End of the list

1. Inserting At Beginning of the list

We can use the following steps to insert a new node at beginning of the double linked list...

Step 1 - Create a **newNode** with given value and **newNode** → **previous** as **NULL**.

Step 2 - Check whether list is **Empty** (**head == NULL**)

Step 3 - If it is **Empty** then, assign **NULL** to **newNode** → **next** and **newNode** to **head**.

Step 4 - If it is **not Empty** then, assign **head** to **newNode** → **next** and **newNode** to **head**.

Linear Data Structure-Linked List

Double Linked List

2. Inserting At End of the list

We can use the following steps to insert a new node at end of the double linked list...

Step 1 - Create a **newNode** with given value and **newNode** → **next** as **NULL**.

Step 2 - Check whether list is **Empty** (**head == NULL**)

Step 3 - If it is **Empty**, then assign **NULL** to **newNode** → **previous** and **newNode** to **head**.

Step 4 - If it is **not Empty**, then, define a node pointer **temp** and initialize with **head**.

Step 5 - Keep moving the **temp** to its next node until it reaches to the last node in the list (until **temp** → **next** is equal to **NULL**).

Step 6 - Assign **newNode** to **temp** → **next** and **temp** to **newNode** → **previous**.

.

Linear Data Structure-Linked List

Double Linked List

❖ **Deletion**

In a double linked list, the deletion operation can be performed in two ways as follows...

1. Deleting from Beginning of the list
2. Deleting from End of the list

1. **Deleting from Beginning of the list**

We can use the following steps to delete a node from beginning of the double linked list...

Step 1 - Check whether list is **Empty** (**head == NULL**)

Step 2 - If it is **Empty** then, display **'List is Empty!!! Deletion is not possible'** and terminate the function.

Step 3 - If it is not Empty then, define a Node pointer **'temp'** and initialize with **head**.

Step 4 - Check whether list is having only one node (**temp → previous** is equal to **temp → next**)

Step 5 - If it is **TRUE**, then set **head** to **NULL** and delete **temp** (Setting **Empty** list conditions)

Step 6 - If it is **FALSE**, then assign **temp → next** to **head**, **NULL** to **head → previous** and delete **temp**.

Double Linked List

2. **Deleting from End of the list**

We can use the following steps to delete a node from end of the double linked list...

Step 1 - Check whether list is **Empty** (**head == NULL**)

Step 2 - If it is **Empty**, then display **'List is Empty!!! Deletion is not possible'** and terminate the function.

Step 3 - If it is not Empty then, define a Node pointer **'temp'** and initialize with **head**.

Step 4 - Check whether list has only one Node (**temp** → **previous** and **temp** → **next** both are **NULL**)

Step 5 - If it is **TRUE**, then assign **NULL** to **head** and delete **temp**. And terminate from the function. (Setting **Empty** list condition)

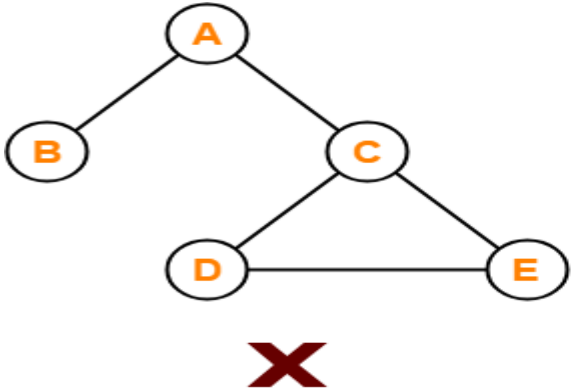
Step 6 - If it is **FALSE**, then keep moving **temp** until it reaches to the last node in the list. (until **temp** → **next** is equal to **NULL**)

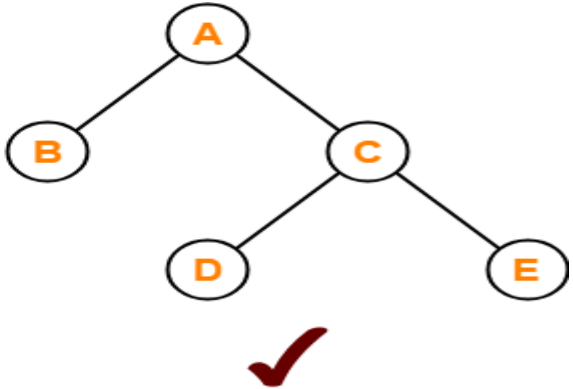
Step 7 - Assign **NULL** to **temp** → **previous** → **next** and delete **temp**.

Non Linear Data Structure

Trees

- Tree is a non-linear data structure which organizes data in a hierarchical structure
- Tree will have special node called ‘root node’
- Tree can have n number of sub node or child node

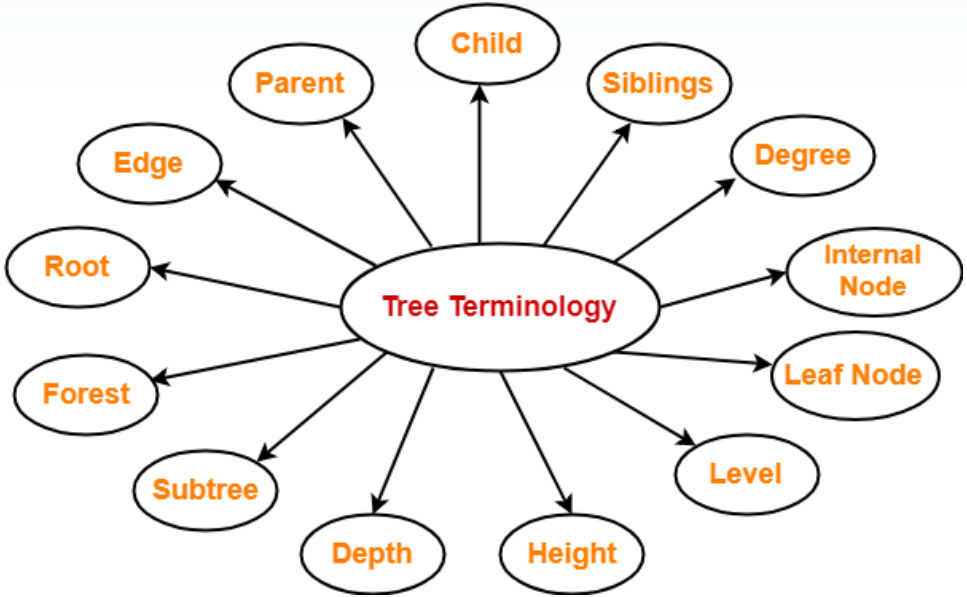




Non Linear Data Structure-Tree

Tree Terminology

The important terms related to tree data structure are-

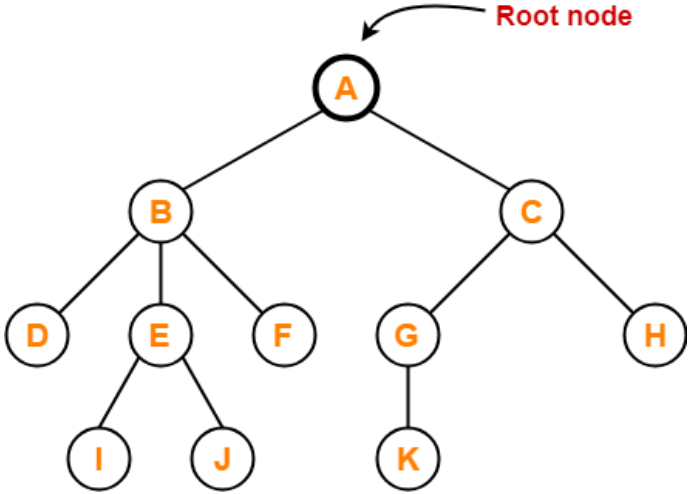


Non Linear Data Structure-Tree

Root

- The first node from where the tree originates is called as a **root node**.
- In any tree, there must be only one root node.
- We can never have multiple root nodes in a tree data structure.

Example



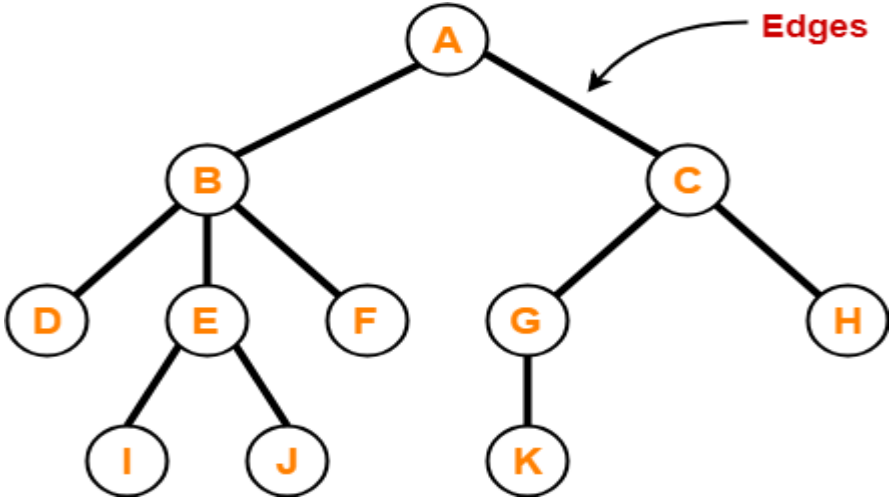
MIDIGIWORLD

Non Linear Data Structure-Tree

Edge

- The connecting link between any two nodes is called as an **edge**.
- In a tree with n number of nodes, there are exactly (n-1) number of edges.

Example

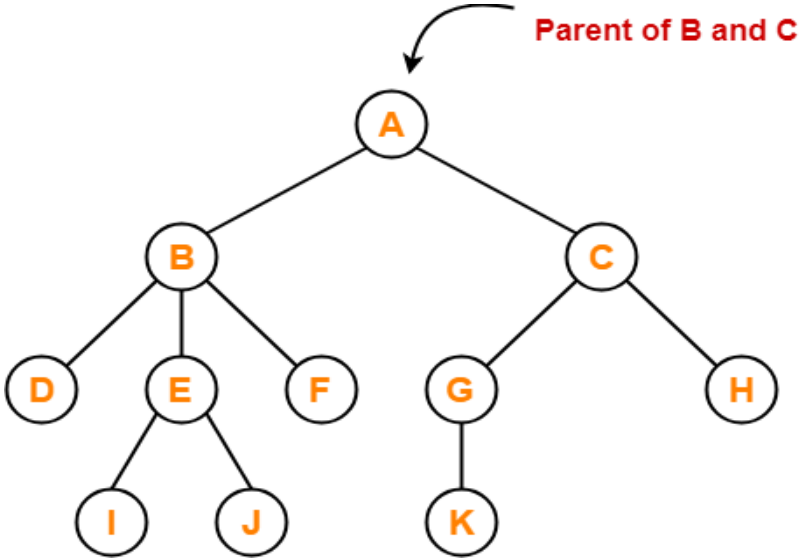


Non Linear Data Structure-Tree

Parent

- The node which has a branch from it to any other node is called as a **parent node**.
- In other words, the node which has one or more children is called as a parent node.
- In a tree, a parent node can have any number of child nodes.

Example

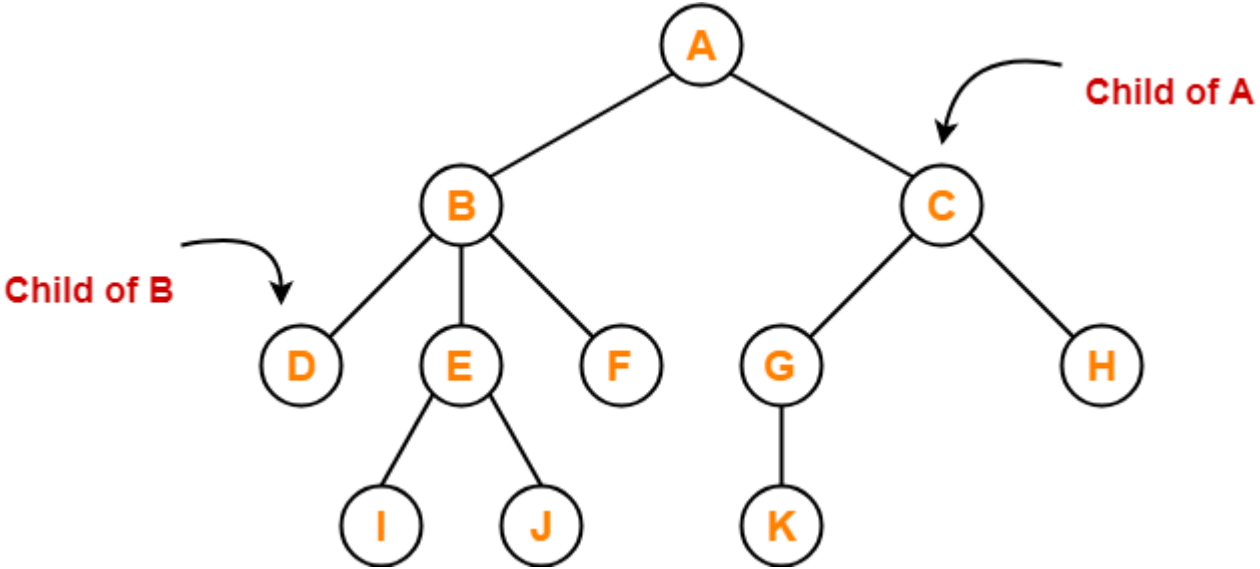


Non Linear Data Structure-Tree

Child

- The node which is a descendant of some node is called as a **child node**.
- All the nodes except root node are child nodes.

Example



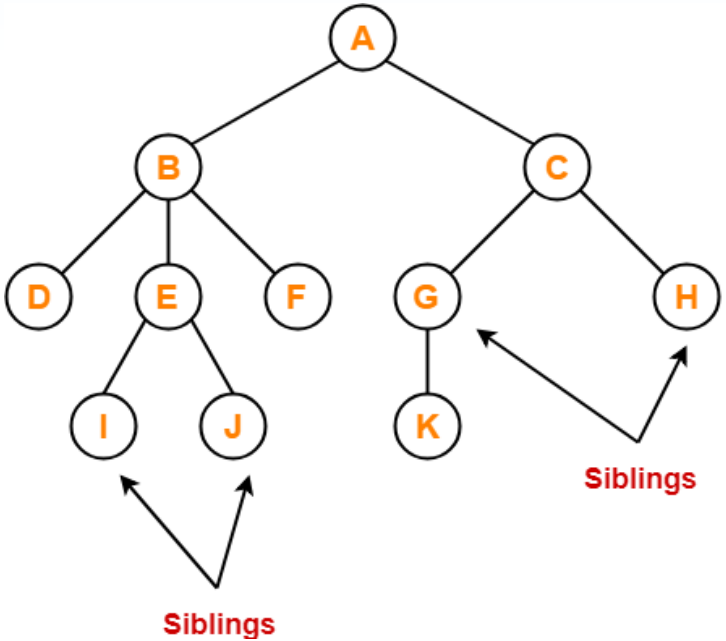
MIDIGIWORLD

Non Linear Data Structure-Tree

Siblings

- Nodes which belong to the same parent are called as **siblings**.
- In other words, nodes with the same parent are sibling nodes.

Example



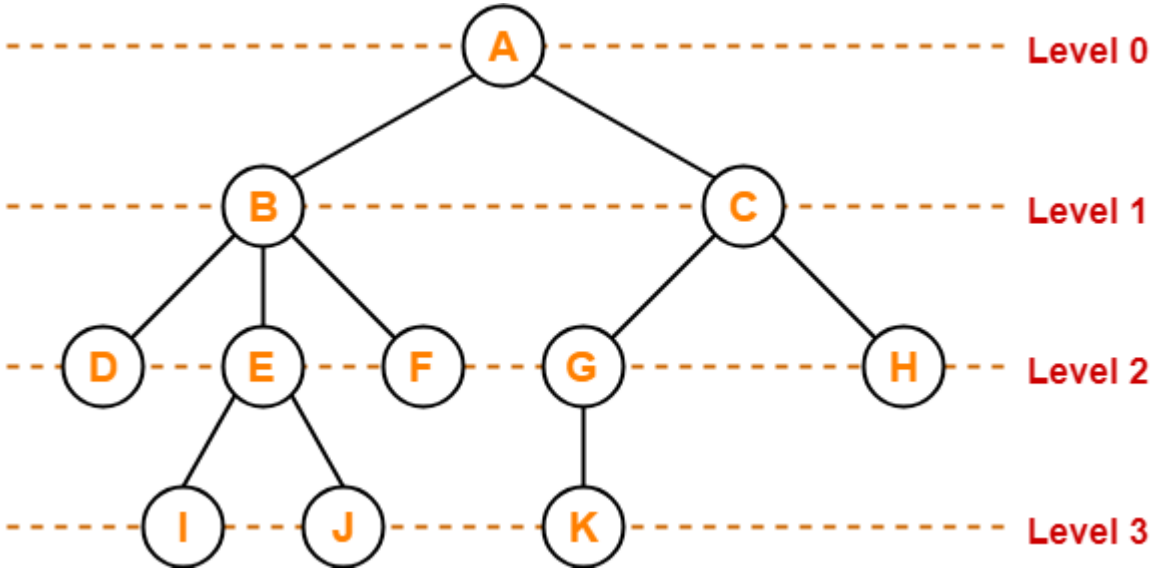
MIDIGIWORLD

Non Linear Data Structure-Tree

Level

- In a tree, each step from top to bottom is called as **level of a tree**.
- The level count starts with 0 and increments by 1 at each level or step.

Example



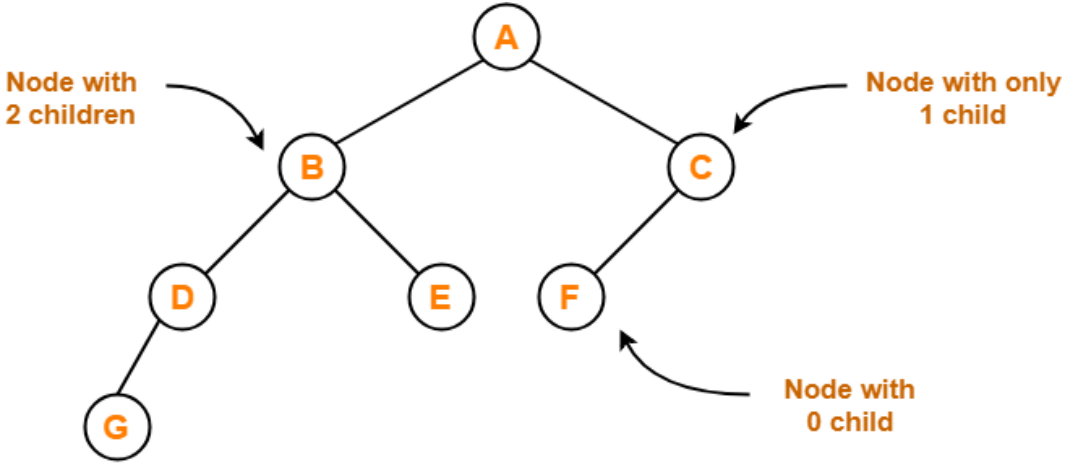
MIDIGIWORLD

Non Linear Data Structure-Tree

Binary Tree

- Binary tree is a special tree data structure in which each node can have at most 2 children.
- Thus, in a binary tree, Each node has either 0 child or 1 child or 2 children.

Example

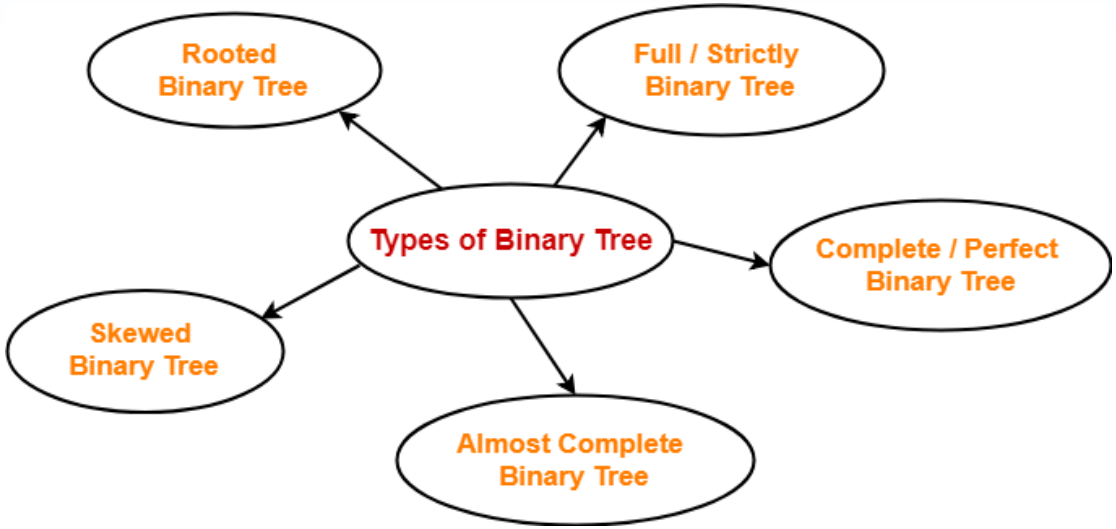


Binary Tree Example

Non Linear Data Structure-Tree

Types of Binary Trees

Binary trees can be of the following types

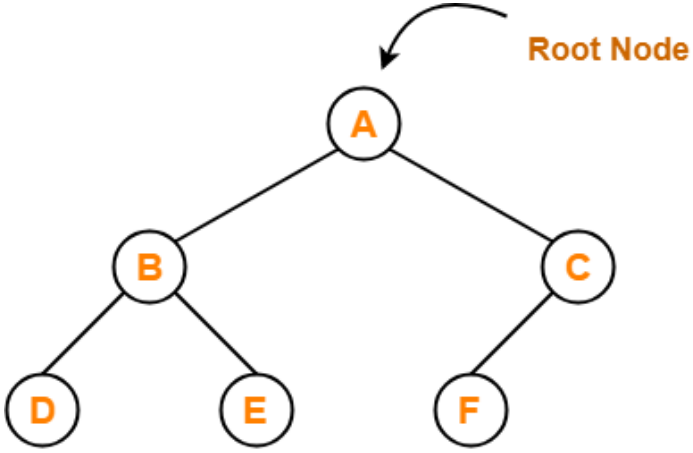


MIDIGIWORLD

Non Linear Data Structure- Tree

Rooted Binary Tree

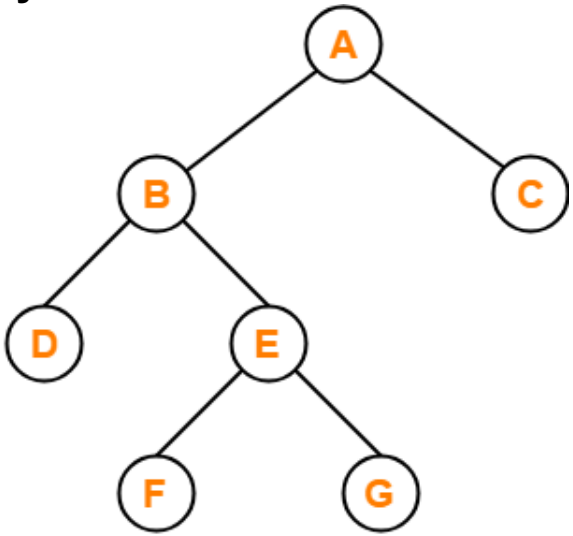
- A **rooted binary tree** is a binary tree that satisfies the following 2 properties-
  1. It has a root node.
  2. Each node has at most 2 children.



Rooted Binary Tree

Strictly Binary Tree

- A binary tree in which every node has either 0 or 2 children is called as a **Full binary tree**.
- Full binary tree is also called as **Strictly binary tree**.



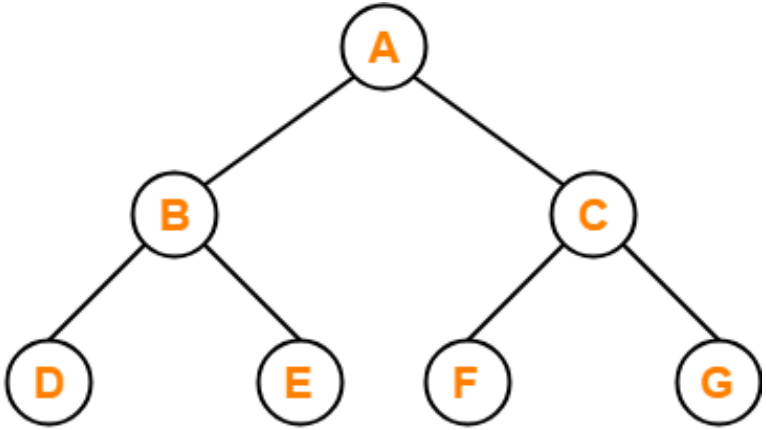
MIDIGIWORLD

Non Linear Data Structure-Tree

Complete Binary Tree

- A **complete binary tree** is a binary tree that satisfies the following 2 properties.

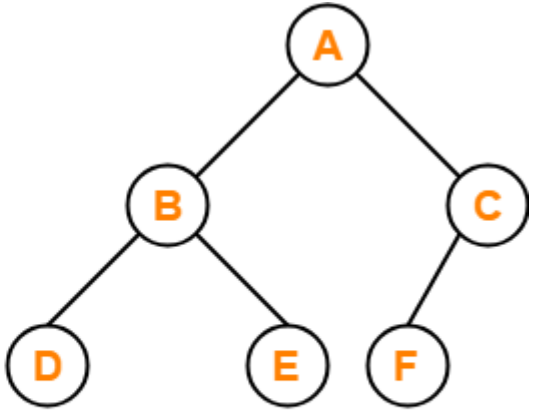
1. Every internal node has exactly 2 children.
2. All the leaf nodes are at the same level.



Almost Complete Binary Tree

- An **almost complete binary tree** is a binary tree that satisfies the following 2 properties.

1. All the levels are completely filled except possibly the last level.
2. The last level must be strictly filled from left to right.



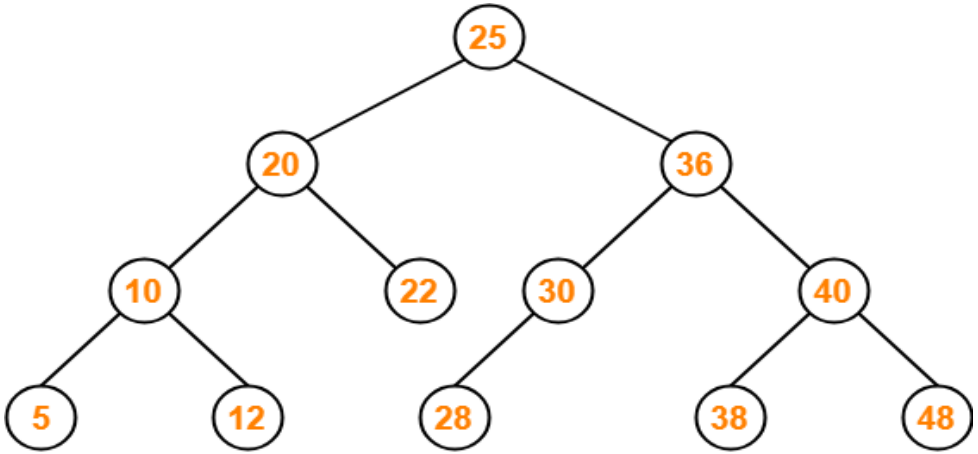
MIDIGIWORLD

Non Linear Data Structure-Tree

Binary Search Tree

- Binary Search Tree is a special kind of binary tree in which nodes are arranged in a specific order.
  1. Binary Search Tree will be constructed based on root Value.
  2. Values which are smaller than root value, will move left side of the root.
  3. Values which are greater than root value, will move right side of the root.

Example



Binary Search Tree

MIDIGIWORLD

Non Linear Data Structure-Tree

Operations on a Binary Search Tree

- The following operations are performed on a binary search tree...

  1. Insertion **2. Traversal –Search & Display**

Insertion Operation in BST

Step 1 - Create a newNode with given value and set its **left** and **right** to **NULL**.

Step 2 - Check whether tree is Empty.

Step 3 - If the tree is **Empty**, then set **root** to **newNode**.

Step 4 - If the tree is **Not Empty**, then check whether the value of newNode is **smaller** or **larger** than the node (here it is root node).

Step 5 - If newNode is **smaller** than **or equal** to the node then move to its **left** child. If newNode is **larger** than the node then move to its **right** child.

Step 6 - Repeat the above steps until we reach to the **leaf** node (i.e., reaches to NULL).

Step 7 - After reaching the leaf node, insert the newNode as **left child** if the newNode is **smaller or equal** to that leaf node or else insert it as **right child**.

Non Linear Data Structure-Tree

Search Operation in BST

Step 1 - Read the search element from the user.

Step 2 - Compare the search element with the value of root node in the tree.

Step 3 - If both are matched, then display "Given node is found!!!" and terminate the function

Step 4 - If both are not matched, then check whether search element is smaller or larger than that node value.

Step 5 - If search element is smaller, then continue the search process in left subtree.

Step 6- If search element is larger, then continue the search process in right subtree.

Step 7 - Repeat the same until we find the exact element or until the search element is compared with the leaf node

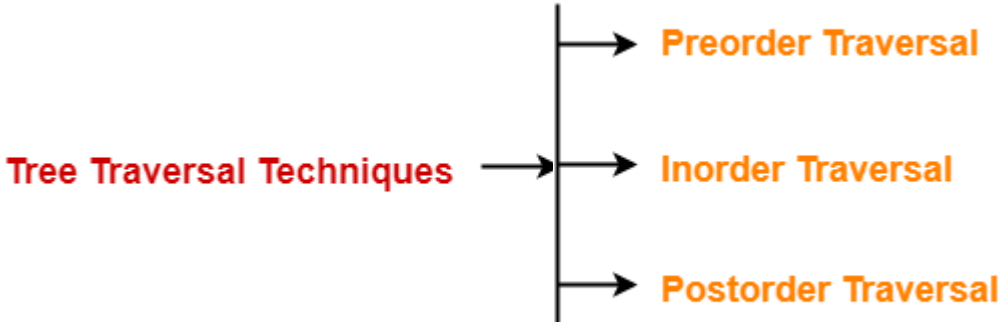
Step 8 - If we reach to the node having the value equal to the search value then display "Element is found" and terminate the function.

Step 9 - If we reach to the leaf node and if it is also not matched with the search element, then display "Element is not found" and terminate the function.

Non Linear Data Structure- Tree

Binary Tree/ Binary Search Tree Traversal

- Tree Traversal refers to the process of visiting each node in a tree data structure exactly once.

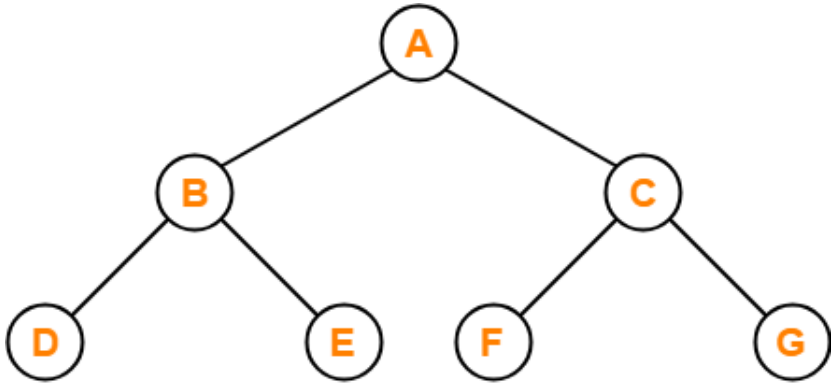


- These three traversal techniques fall under Depth First Traversal

Non Linear Data Structure-Tree

Pre-order Traversal

- Steps involved in the Process of Pre-Order Traversal.
  1. Visit the root
  2. Traverse the left sub tree
  3. Traverse the right sub tree



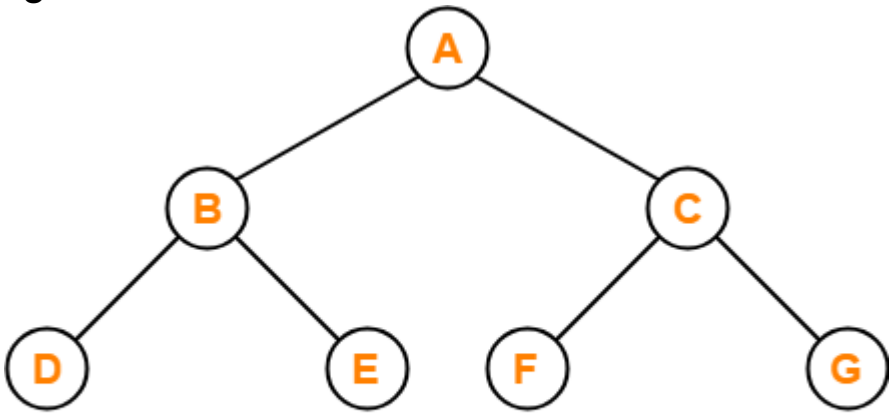
Preorder Traversal : A , B , D , E , C , F , G

MIDIGIWORLD

Non Linear Data Structure-Tree

In-order Traversal

- Steps involved in the Process of Pre-Order Traversal.
  1. Traverse the left sub tree
  2. Visit the root
  3. Traverse the right sub tree

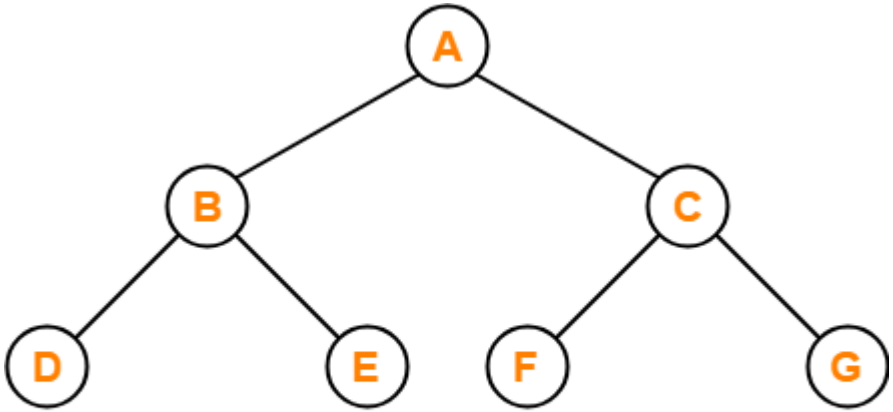


Inorder Traversal : D , B , E , A , F , C , G

Non Linear Data Structure-Tree

Post-order Traversal

- Steps involved in the Process of Pre-Order Traversal.
  1. Traverse the left sub tree
  2. Traverse the right sub tree
  3. Visit the root



Postorder Traversal : D , E , B , F , G , C , A